

LAPORAN PROJECT SATU KAMSIS

OLEH:

**IBNU SUGENG MUGIANA
101032300124 – TK-47-03**

**TELKOM UNIVERSITY
FAKULTAS TEKNIK ELEKTRO
JURUSAN S1 TEKNIK KOMPUTER
MARET 2026**



Telkom University

**PROGRAM STUDI S1 TEKNIK KOMPUTER
FAKULTAS TEKNIK ELEKTRO
UNIVERSITAS TELKOM
BANDUNG
2026**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Keamanan sistem merupakan aspek yang sangat penting dalam era digital saat ini. Seiring meningkatnya penggunaan sistem berbasis jaringan dan pertukaran data secara elektronik, ancaman terhadap kerahasiaan, integritas, dan ketersediaan informasi juga semakin meningkat. Salah satu teknik yang paling banyak digunakan untuk melindungi kerahasiaan data adalah enkripsi.

Algoritma AES (Advanced Encryption Standard) merupakan standar enkripsi simetris yang ditetapkan oleh NIST (National Institute of Standards and Technology) pada tahun 2001. AES menggantikan algoritma DES (Data Encryption Standard) yang dinilai sudah tidak aman akibat panjang kuncinya yang terlalu pendek. AES-128 merupakan varian AES dengan panjang kunci 128 bit dan terbukti memberikan keamanan yang sangat tinggi terhadap berbagai serangan kriptanalisis.

Mode CBC (Cipher Block Chaining) adalah salah satu mode operasi AES yang paling sering digunakan karena memberikan keamanan yang lebih baik dibandingkan mode ECB (Electronic CodeBook). Pada mode CBC, setiap blok plaintext di-XOR dengan blok ciphertext sebelumnya sebelum dienkripsi, sehingga pola yang sama pada plaintext tidak menghasilkan pola yang sama pada ciphertext.

1.2 Tujuan

Tujuan dari Mata Kuliah ini adalah sebagai berikut:

1. Memahami prinsip dasar algoritma enkripsi AES-128.
2. Memahami cara kerja mode operasi CBC (Cipher Block Chaining).
3. Mengimplementasikan proses enkripsi dan dekripsi AES-128 CBC menggunakan JavaScript.
4. Membuat aplikasi enkripsi berbasis web yang interaktif dan mudah digunakan.
5. Menganalisis kelemahan dan kelebihan mode CBC dibandingkan mode ECB.

1.3 Rumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah yang dibahas dalam pelajaran ini adalah:

1. Bagaimana cara kerja algoritma AES-128 dalam proses enkripsi dan dekripsi?
2. Bagaimana implementasi mode CBC pada algoritma AES-128 menggunakan JavaScript?
3. Mengapa mode CBC lebih direkomendasikan dibandingkan mode ECB dalam aplikasi nyata?

1.4 Batasan Masalah

Batasan masalah dalam pembelajaran ini meliputi:

1. Algoritma yang diimplementasikan adalah AES-128 (kunci 128-bit).
2. Mode operasi yang digunakan adalah CBC dan ECB.

3. Implementasi menggunakan JavaScript murni tanpa library eksternal.
4. Input data berupa teks (plaintext) dan output berupa hexadecimal.

BAB II

DASAR TEORI

2.1 Kriptografi

Kriptografi adalah ilmu dan seni untuk mengamankan pesan dengan cara mengubah pesan asli (plaintext) menjadi bentuk yang tidak dapat dibaca (ciphertext) sehingga hanya pihak yang berwenang yang dapat memahaminya. Kriptografi merupakan komponen utama dalam keamanan informasi modern dan digunakan secara luas dalam berbagai aplikasi seperti komunikasi aman, penyimpanan data, dan autentikasi digital.

2.2 Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) adalah algoritma enkripsi simetris yang diadopsi sebagai standar enkripsi oleh pemerintah Amerika Serikat. AES menggunakan panjang blok tetap 128 bit dan mendukung tiga ukuran kunci: 128 bit, 192 bit, dan 256 bit. Pada teori ini digunakan AES-128 dengan panjang kunci 128 bit.

AES bekerja pada sebuah matriks 4x4 byte yang disebut state. Proses enkripsi AES terdiri dari beberapa putaran (rounds), di mana setiap putaran menerapkan empat transformasi utama:

1. SubBytes: Setiap byte pada state digantikan menggunakan tabel substitusi (S-Box).
2. ShiftRows: Baris-baris pada matriks state digeser secara siklik.
3. MixColumns: Setiap kolom dikalikan dengan matriks konstan dalam $GF(2^8)$.
4. AddRoundKey: State di-XOR dengan round key yang diturunkan dari key expansion.

2.3 Mode Operasi CBC (Cipher Block Chaining)

Mode CBC adalah mode operasi blok cipher di mana setiap blok plaintext di-XOR terlebih dahulu dengan blok ciphertext sebelumnya sebelum dienkripsi. Untuk blok pertama, digunakan Initialization Vector (IV) karena belum ada blok ciphertext sebelumnya.

Proses enkripsi CBC dapat dirumuskan sebagai berikut: $C[i] = E(K, P[i] \text{ XOR } C[i-1])$, di mana $C[0] = IV$. Sedangkan proses dekripsi: $P[i] = D(K, C[i]) \text{ XOR } C[i-1]$. Kelebihan mode CBC adalah pola data yang sama pada plaintext tidak akan menghasilkan pola yang sama pada ciphertext, sehingga lebih aman dari mode ECB.

2.4 Initialization Vector (IV)

Initialization Vector (IV) adalah nilai acak yang digunakan pada awal proses enkripsi CBC. IV berfungsi untuk memastikan bahwa dua pesan yang identik menghasilkan ciphertext yang berbeda apabila dienkripsi dengan kunci yang sama. Nilai IV tidak perlu dirahasiakan dan umumnya dikirimkan bersama ciphertext.

BAB III

HASIL DAN PEMBAHASAN

3.1 Deskripsi Aplikasi

Aplikasi enkripsi data berbasis web yang dibuat terdiri dari sebuah file HTML tunggal (enkripsi.html) yang memuat antarmuka pengguna sekaligus logika enkripsi-dekripsi. Aplikasi berjalan sepenuhnya di browser tanpa memerlukan koneksi server maupun library eksternal.

Antarmuka aplikasi menyediakan input untuk teks/pesan (plaintext atau ciphertext dalam hex), kunci rahasia, Initialization Vector (IV), serta pilihan mode algoritma (CBC atau ECB). Terdapat dua tombol aksi utama, yaitu Enkripsi dan Dekripsi, serta tombol Copy Output untuk menyalin hasil ke clipboard.

3.2 Struktur Kode Program

Kode program dibagi menjadi dua bagian utama, yaitu HTML/CSS untuk tampilan antarmuka dan JavaScript untuk logika enkripsi. Pada bagian JavaScript, diimplementasikan sebuah class bernama AES128 yang mengenkapsulasi seluruh fungsi enkripsi.

Komponen utama class AES128 adalah sebagai berikut:

Komponen	Fungsi
sbox	Tabel substitusi (S-Box) AES 256 entri untuk operasi SubBytes
inv_sbox	Invers S-Box untuk operasi InvSubBytes pada dekripsi
rcon	Round Constant yang digunakan dalam proses key expansion
generateIV()	Menghasilkan IV acak 16 byte menggunakan Math.random()
keyExpansion()	Mengembangkan kunci 128-bit menjadi 44 word round keys
subBytes() / invSubBytes()	Substitusi byte menggunakan S-Box dan Invers S-Box
shiftRows() / invShiftRows()	Penggeseran baris pada matriks state
mixColumns() / invMixColumns()	Pencampuran kolom menggunakan operasi GF(2 ⁸)
addRoundKey()	XOR state dengan round key yang sesuai
encryptBlock() / decryptBlock()	Enkripsi/dekripsi satu blok 16 byte
pkcs7Pad() / pkcs7Unpad()	Penambahan/penghapusan PKCS#7 padding
encryptCBC() / decryptCBC()	Enkripsi/dekripsi penuh menggunakan mode CBC

Tabel 3.1 Komponen utama class AES128

3.3 Proses Enkripsi

Ketika pengguna menekan tombol Enkripsi, fungsi `encrypt()` dipanggil. Fungsi ini mengambil nilai dari field kunci, plaintext, dan IV. Kunci dipadding otomatis menjadi 16 karakter jika kurang dari 16. IV di-generate secara acak apabila tidak diisi oleh pengguna.

Selanjutnya, fungsi `encryptCBC()` dipanggil dengan parameter plaintext, kunci yang telah dipadding, dan IV. Fungsi ini terlebih dahulu menerapkan PKCS#7 padding pada plaintext, kemudian memproses setiap blok 16 byte secara berurutan. Setiap blok di-XOR dengan blok ciphertext sebelumnya (atau IV untuk blok pertama) sebelum dienkripsi menggunakan `encryptBlock()`.

Hasil akhir enkripsi berupa byte array yang dikonversi ke format hexadecimal menggunakan fungsi `bytesToHex()` dan ditampilkan pada field output.

3.4 Proses Dekripsi

Proses dekripsi merupakan kebalikan dari enkripsi. Fungsi `decrypt()` mengambil ciphertext dalam format hexadecimal dari field input, kemudian memanggil fungsi `decryptCBC()`. Fungsi ini mengambil 16 byte pertama sebagai IV, lalu mendekripsi setiap blok dan meng-XOR hasilnya dengan blok ciphertext sebelumnya untuk mendapatkan plaintext asli.

Setelah seluruh blok didekripsi, padding PKCS#7 dihapus menggunakan fungsi `pkcs7Unpad()` dan plaintext asli ditampilkan pada field output.

3.5 Hasil Pengujian

Pengujian dilakukan dengan skenario sebagai berikut:

No.	Skenario Pengujian	Input	Hasil
1	Enkripsi teks pendek	Teks: "Hello", Key: "password123"	Ciphertext hex 32 karakter (1 blok + IV)
2	Enkripsi teks panjang	Teks > 16 karakter, Key: "kunciRahasia123"	Ciphertext hex sesuai panjang data + padding
3	Dekripsi hasil enkripsi	Ciphertext dari skenario 1	Berhasil mendekripsi kembali ke "Hello"
4	Validasi kunci minimum	Kunci < 8 karakter	Muncul pesan error: kunci minimal 8 karakter
5	IV kustom	IV: "1234567890abc"	Enkripsi menggunakan IV yang ditentukan

Tabel 3.2 Skenario dan hasil pengujian aplikasi enkripsi

3.6 Pembahasan

Berdasarkan hasil pengujian, aplikasi enkripsi berjalan dengan baik untuk semua skenario yang diuji. Proses enkripsi dan dekripsi berhasil dilakukan secara simetris, artinya plaintext yang dienkripsi dapat didekripsi kembali menggunakan kunci dan IV yang sama.

Validasi input berjalan dengan benar, termasuk pengecekan panjang minimum kunci (8 karakter) dan deteksi input kosong. Fitur auto-generate IV juga berfungsi dengan baik ketika pengguna tidak memasukkan IV secara manual.

Mode CBC terbukti lebih unggul dibandingkan ECB karena menggunakan IV dan XOR antar blok, sehingga menghasilkan distribusi ciphertext yang lebih acak dan tidak memperlihatkan pola data asli. Ini sangat penting untuk keamanan sistem yang mengandung pola berulang.

Namun, terdapat beberapa keterbatasan implementasi ini. Implementasi AES ini tidak menggunakan hardware acceleration atau Web Crypto API, sehingga performanya relatif lebih lambat dibandingkan implementasi native. Selain itu, S-Box yang digunakan perlu diverifikasi ulang untuk memastikan kecocokan dengan spesifikasi AES resmi.

BAB IV

KESIMPULAN DAN SARAN

4.1 Kesimpulan

Berdasarkan pelajaran yang telah dilaksanakan, dapat ditarik kesimpulan sebagai berikut:

1. Algoritma AES-128 CBC berhasil diimplementasikan menggunakan JavaScript murni berbasis web tanpa library eksternal.
2. Mode CBC memberikan keamanan yang lebih baik dibandingkan ECB karena setiap blok bergantung pada blok sebelumnya dan menggunakan IV yang unik.
3. PKCS#7 padding memungkinkan enkripsi data dengan panjang sembarang menggunakan cipher blok tetap.
4. Aplikasi web interaktif berhasil dibuat dan mampu menjalankan enkripsi/dekripsi secara real-time di browser.

4.2 Saran

Beberapa saran untuk pengembangan lebih lanjut:

1. Menggunakan Web Crypto API bawaan browser untuk performa dan keamanan yang lebih baik.
2. Menambahkan fitur enkripsi file (bukan hanya teks) untuk kegunaan yang lebih luas.
3. Menambahkan mode AES-GCM untuk autentikasi data (Authenticated Encryption).
4. Melakukan verifikasi kebenaran S-Box dengan referensi standar FIPS 197.

DAFTAR PUSTAKA

NIST. (2001). Advanced Encryption Standard (AES). FIPS Publication 197. National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.FIPS.197>

Stallings, W. (2017). Cryptography and Network Security: Principles and Practice (7th ed.). Pearson Education.

Menezes, A. J., van Oorschot, P. C., & Vanstone, S. A. (1996). Handbook of Applied Cryptography. CRC Press.

Daemen, J., & Rijmen, V. (2002). The Design of Rijndael: AES – The Advanced Encryption Standard. Springer-Verlag.

RFC 5652. (2009). Cryptographic Message Syntax (CMS). Internet Engineering Task Force (IETF).